Using the `DtlsDatagramChannel` class from the GitHub repository accompanying the book you can now implement a working DTLS client example application. The sample class requires that the underlying UDP channel is *connected* before the DTLS handshake occurs. This restricts the channel to send packets to only a single host and receive packets from only that host too. This is not a limitation of DTLS but just a simplification made to keep the sample code short. A consequence of this decision is that the server that you'll develop in the next section can handle only a single client at a time and will discard packets from other clients. It's not much harder to handle concurrent clients but you need to associate a unique `SSLEngine` with each client.

> **DEFINITION** A UDP channel (or socket) is ***connected*** when it is restricted to only send or receive packets from a single host. Using connected channels simplifies programming and can be more efficient, but packets from other clients will be silently discarded. The `connect()` method is used to connect a Java `DatagramChannel`.

Listing 12.3 shows a sample client that connects to a server and then sends the contents of a text file line by line. Each line is sent as an individual UDP packet and will be encrypted using DTLS. After the packets are sent, the client queries the `SSLSession` to print out the DTLS cipher suite that was used for the connection. Open the DtlsClient.java file you created earlier and add the main method shown in the listing. Create a text file named test.txt in the root folder of the project and add some example text to it such as lines